

Notations

- On appelle machine tout triplet (Q, Σ, δ) où Q est un ensemble fini non vide dont les éléments sont appelés états, Σ un ensemble fini non vide appelé alphabet dont les éléments sont appelés lettres, et δ une application de $Q \times \Sigma$ dans Q appelée fonction de transition. Une machine correspond donc à un automate déterministe complet sans notion d'état initial ou d'états finaux.
- Pour un état q et une lettre x , on note $q.x = \delta(q, x)$.
- L'ensemble des mots (c'est-à-dire des concaténations de lettres) sur l'alphabet Σ est noté Σ^* .
- Le mot vide est noté ε .
- On note ux le mot obtenu par la concaténation du mot u et de la lettre x .
- On note δ^* l'extension à $Q \times \Sigma^*$ de la fonction de transition δ définie par

$$\begin{cases} \forall q \in Q \\ \delta^*(q, \varepsilon) = q \\ \forall (q, x, u) \in Q \times \Sigma \times \Sigma^* \\ \delta^*(q, xu) = \delta^*(\delta(q, x), u) \end{cases}$$

- Pour un état q de Q et un mot m de Σ^* , on note encore $q.m$ pour désigner $\delta^*(q, m)$.
- Pour deux états q et q' , on dit que q' est accessible depuis q s'il existe un mot u tel que $q' = q.u$.
- On dit qu'un mot m de Σ^* est synchronisant pour une machine (Q, Σ, δ) s'il existe un état q_0 de Q tel que pour tout état q de Q , $q.m = q_0$.

L'existence de tels mots dans certaines machines est utile car elle permet de ramener une machine dans un état particulier connu en lisant un mot donné (donc en pratique de la « réinitialiser » par une succession précise d'ordres passés à la machine réelle).

La partie I étudie quelques considérations générales sur les mots synchronisants, la partie II est consacrée à des problèmes algorithmiques classiques, la partie III relie le problème de la satisfiabilité d'une formule logique à celui de la recherche d'un mot synchronisant de longueur donnée dans une certaine machine, et la partie IV étudie l'existence d'un mot synchronisant pour une machine donnée. Les parties I, II et III peuvent être traitées indépendamment. La partie IV, plus technique, utilise la partie II.

Dans les exemples concrets de machines donnés plus loin, l'ensemble d'états peut être quelconque, de même que l'alphabet ($\Sigma = \{0, 1\}, \{a, b, c\}, \dots$). Pour la modélisation en OCaml, l'alphabet Σ sera toujours considéré comme étant un intervalle d'entiers $\llbracket 0, p-1 \rrbracket$ où $p = |\Sigma|$. Une lettre correspondra donc à un entier entre 0 et $p-1$. Un mot de Σ^* sera représenté par une liste de lettres (donc d'entiers).

```
type lettre = int
type mot = lettre list
```

De même, en OCaml, l'ensemble d'états Q d'une machine sera toujours considéré comme étant l'intervalle d'entiers $\llbracket 0, n-1 \rrbracket$ où $n = |Q|$.

```
type etat = int
```

Ainsi, la fonction de transition δ d'une machine sera modélisée par une fonction OCaml de signature `etat -> lettre -> etat`. On introduit alors le type :

```
type machine =
  {n_etats : int;
   n_lettres : int;
   delta : etat -> lettre -> etat}
```

`n_etats` correspond au cardinal de Q , `n_lettres` à celui de Σ et `delta` à la fonction de transition. Dans tout le problème, on supposera que `m.delta` s'exécute en temps constant.

Par exemple, on peut créer une machine `m0` à trois états sur un alphabet à deux lettres ayant comme fonction de transition la fonction `f0` donnée ci-après :

```
let f0 etat lettre =
  let matrice =
```

```

[| [|1; 1|];
 [|0; 2|];
 [|0; 2|] |] in
matrice.(etat).(lettre)

```

```

let m0 = {n_etats = 3; n_lettres = 2; delta = f0}

```

La figure 1 fournit une représentation de la machine M_0 .

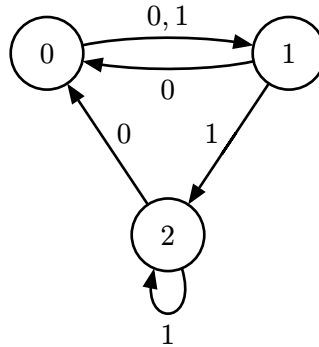


Figure 1 : la machine M_0 .

On pourra observer que les mots 11 et 10 sont tous les deux synchronisants pour la machine M_0 .

Dans tout le sujet, si une question demande la complexité d'un programme ou d'un algorithme, on attend une complexité temporelle exprimée en $O(\dots)$.

I – Considérations générales

1. Que dire des mots synchronisants pour une machine ayant un seul état ?

Dans toute la suite du problème, on supposera que les machines ont au moins deux états.

2. On considère la machine M_1 représentée figure 2. Donner un mot synchronisant pour M_1 s'il en existe un. Justifier la réponse.

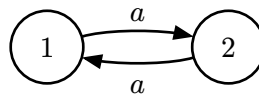


Figure 2 : la machine M_1 .

3. On considère la machine M_2 représentée figure 3. Donner un mot synchronisant de trois lettres pour M_2 . On ne demande pas de justifier la réponse.

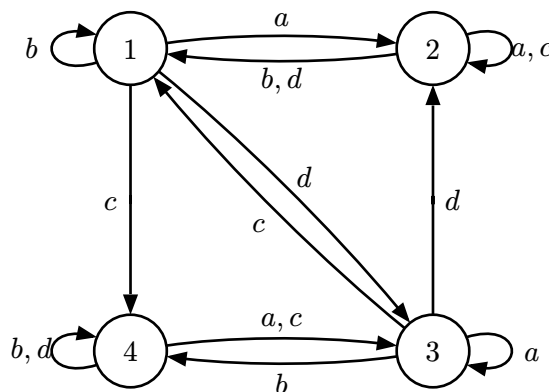


Figure 3 : la machine M_2 .

4. Écrire une fonction `delta_etoile` qui, prenant en entrée une machine M , un état q et un mot u , renvoie l'état atteint par la machine M en partant de l'état q et en lisant le mot u .

```

val delta_etoile : machine -> etat -> mot -> etat

```

5. Écrire une fonction `est_synchronisant` qui, prenant en entrée une machine M et un mot u , dit si le mot est synchronisant pour M .

```
val est_synchronisant : machine -> mot -> bool
```

6. Montrer que pour qu'une machine ait un mot synchronisant, il faut qu'il existe une lettre x et deux états distincts de Q , q et q' , tels que $q.x = q'.x$.

Soit $LS(M)$ le langage des mots synchronisants d'une machine $M = (Q, \Sigma, \delta)$. On introduit la machine des parties $\hat{M} = (\hat{Q}, \Sigma, \hat{\delta})$ où $\hat{Q}$ est l'ensemble des parties de Q et où $\hat{\delta}$ est définie par

$$\forall P \subseteq Q, \quad \forall x \in \Sigma, \quad \hat{\delta}(P, x) = \{\delta(p, x) \mid p \in P\}.$$

7. Justifier que l'existence d'un mot synchronisant pour M se ramène à un problème d'accessibilité de certain(s) état(s) depuis certain(s) état(s) dans la machine des parties.
8. En déduire que le langage $LS(M)$ des mots synchronisants de la machine M est reconnaissable.
9. Déterminer la machine des parties associée à la machine M_0 , puis donner une expression régulière du langage $LS(M_0)$.
10. Montrer que si l'on sait résoudre le problème de l'existence d'un mot synchronisant, on sait dire, pour une machine M et un état q_0 de M choisi, s'il existe un mot u tel que pour tout état q de Q , le chemin menant de q à $q.u$ passe forcément par q_0 .

II – Algorithmes classiques

On appellera graphe d'automate tout couple (S, A) où S est un ensemble dont les éléments sont appelés sommets et A une partie de $S \times \Sigma \times S$ dont les éléments sont appelés arcs. Pour un arc (q, x, q') , x est l'étiquette de l'arc, q son origine et q' son extrémité. Un graphe d'automate correspond donc à un automate non déterministe sans notion d'état initial ou final.

Soient s et s' deux sommets d'un graphe (S, A) . On appelle chemin de s vers s' de longueur ℓ toute suite d'arcs $(s_1, x_1, s_{1'}), \dots, (s_\ell, x_\ell, s_{\ell'})$ de A telle que $s_1 = s$, $s_{\ell'} = s'$ et, pour tout $i \in \llbracket 1, \ell - 1 \rrbracket$, $s_{i'} = s_{i+1}$. L'étiquette de ce chemin est alors le mot $x_1 \dots x_\ell$ et on dit que s' est accessible depuis s . En particulier, pour tout $s \in S$, s est accessible depuis s par le chemin vide d'étiquette ε .

Par exemple, avec

$$\Sigma = \{a, b\}, \quad S_0 = \{0, 1, 2, 3, 4, 5\},$$

$$A_0 = \{(0, b, 0), (0, a, 3), (0, b, 2), (0, a, 1), (1, a, 1), (1, a, 2), (2, b, 1), \\ (2, b, 3), (2, b, 4), (3, a, 2), (4, a, 1), (4, b, 5), (5, a, 1)\}.$$

le graphe d'automate $G_0 = (S_0, A_0)$ est représenté en figure 4.

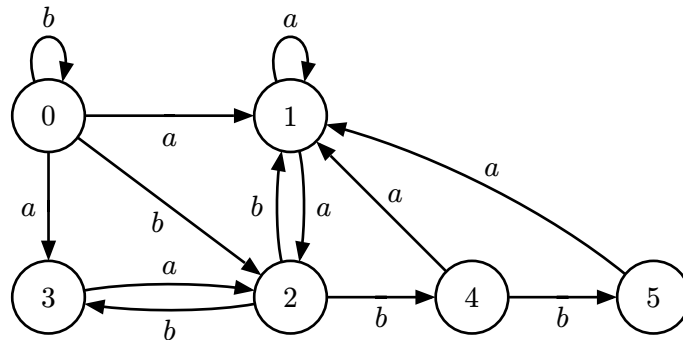


Figure 4 : le graphe d'automate G_0 .

Dans les programmes à écrire, un graphe aura toujours pour ensemble de sommets un intervalle d'entiers $\llbracket 0, n - 1 \rrbracket$ et l'ensemble des arcs étiquetés par Σ (supposé être un intervalle $\llbracket 0, p - 1 \rrbracket$) sera codé par un tableau de listes d'adjacence v : pour tout $s \in S$, $v.(s)$ est la liste (dans n'importe quel ordre) de tous les couples (s', x) tels que (s, x, s') soit un arc du graphe. Pour des raisons de compatibilité ultérieure, les sommets seront codés par le type `etat`.

Avec l'alphabet $\Sigma = \{a, b\}$, la lettre a est codée 0 et la lettre b est codée 1. L'ensemble des arcs de G_0 , dont chaque sommet est codé par son numéro, admet pour représentation OCaml :

```
let v0 : (etat * lettre) list array =
  [| [(0,1);(3,0);(2,1);(1,0)];
    [(1,0);(2,0)];
    [(1,1);(3,1);(4,1)];
    [(2,0)];
    [(1,0);(5,1)];
    [(1,0)] |]
```

On veut implémenter une file d'attente à l'aide d'un tableau circulaire, dans lequel les indices sont considérés modulo la longueur du tableau. On définit pour cela un type :

```
type 'a file = {
  tab : 'a array;
  mutable deb : int;
  mutable fin : int;
  mutable vide : bool
}
```

- `deb` indique l'indice du premier (plus ancien) élément de la file.
- `fin` indique l'indice suivant (modulo la longueur du tableau) celui du dernier (plus récent) élément de la file.
- Il est tout à fait possible d'avoir `deb > fin`.

11. Expliquer pourquoi le champ `vide` est nécessaire.

12. Écrire une fonction `ajoute` telle que `ajoute f x` ajoute `x` à la fin de la file d'attente `f`. Si c'est impossible, la fonction lèvera une exception.

```
val ajoute : 'a file -> 'a -> unit
```

13. Écrire une fonction `retire` telle que `retire f` retire l'élément en tête de la file d'attente et le renvoie. Si c'est impossible, la fonction lèvera une exception.

```
val retire : 'a file -> 'a
```

14. Quelle est la complexité de ces fonctions ?

On considère l'algorithme 1 suivant s'appliquant à un graphe d'automate $G = (S, A)$ et à un ensemble de sommets X (on note $n = |S|$ et ∞ , « vide » et « rien » des valeurs particulières).

Algorithme 1

```
F ← file vide
D ← (∞, ..., ∞) (tableau de longueur n)
P ← (vide, ..., vide) (tableau de longueur n)
c ← n
Pour s ∈ X :
  Ajouter s à F
  D[s] ← 0
  P[s] ← rien
  c ← c - 1
Tant que F n'est pas vide :
  s ← Retirer de F
  Pour (s, y, s') un arc de A tel que D[s] = ∞ :
    D[s'] ← D[s] + 1
    P[s'] ← (s, y)
  Ajouter s' à F
```

```

    c ← c - 1
    Renvoyer (c, D, P)

```

15. Justifier que l'algorithme 1 termine toujours.
 16. Donner la complexité de cet algorithme en fonction de $|S|$ et $|A|$. Justifier la réponse.
 17. Justifier qu'au début de chaque passage dans la boucle « tant que F n'est pas vide », si F contient dans l'ordre les sommets $s_1, s_2, \dots, s_r$, alors $D[s_1] \leq D[s_2] \leq \dots \leq D[s_r]$ et $D[s_r] - D[s_1] \leq 1$.
- Pour s sommet de G , on note d_s la distance de X à s , c'est-à-dire la longueur d'un plus court chemin d'un sommet de X à s (avec la convention $d_s = \infty$ s'il n'existe pas de tel chemin).
18. Justifier brièvement qu'à la fin de l'algorithme, pour tout sommet s , $D[s] \neq \infty$ si et seulement si s est accessible depuis un sommet de X et que $d_s \leq D[s]$. Que désigne alors c ?
 19. Montrer qu'en fait, à la fin, on a pour tout sommet s , $D[s] = d_s$. Que vaut alors $P[s]$?
 20. Écrire une fonction `accessibles` prenant en entrée un graphe d'automate (sous forme de son tableau de listes d'adjacence `v`) et un ensemble `x` de sommets (sous forme d'une liste d'états) et renvoyant le triplet (c, D, P) calculé selon l'algorithme précédent. La constante ∞ sera représentée par l'entier -1 , et pour les couples (état, lettre) on utilisera le type suivant :

```

type arc = Rien | Vide | Arc of état * lettre

val accessibles : ((état * lettre) list) array -> état list
                -> int * int array * arc array

```

21. Écrire une fonction `chemin` qui, prenant en entrée un sommet `s` et le tableau `p` calculé à l'aide de la fonction `accessibles` sur un graphe G et un ensemble X , renvoie un mot de longueur minimale qui est l'étiquette d'un chemin d'un sommet de X à s (ou lèvera une exception s'il n'en existe pas).

```

val chemin : état -> arc array -> mot

```

III – Réduction SAT

On s'intéresse à la satisfiabilité d'une formule logique portant sur des variables propositionnelles $x_1, \dots, x_m$. On note $\wedge$ le connecteur logique « et », $\vee$ le connecteur « ou » et $\bar{f}$ la négation d'une formule f . On appelle littéral une formule constituée d'une variable x_i ou de sa négation $\bar{x}_i$, et clause une disjonction de littéraux. Une formule sous forme normale conjonctive est une conjonction de clauses. Par exemple,

$$F_1 = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_4) \wedge (x_2 \vee \bar{x}_3 \vee x_4)$$

est une formule formée de trois clauses et portant sur quatre variables propositionnelles x_1, x_2, x_3, x_4 .

Soit F une formule sous forme normale conjonctive composée de n clauses et faisant intervenir m variables. On suppose les clauses numérotées $c_1, \dots, c_n$. On veut ramener le problème de sa satisfiabilité au problème de la recherche d'un mot synchronisant de longueur inférieure ou égale à m sur une certaine machine. On introduit la machine suivante associée à F :

- Q est formé de $mn + n + 1$ états : un état particulier f et $n(m + 1)$ autres états notés $q_{i,j}$, avec $(i, j) \in \llbracket 1, n \rrbracket \times \llbracket 1, m + 1 \rrbracket$.
- $\Sigma = \{0, 1\}$.
- δ est définie par :
 - f est un état puits : $\delta(f, 0) = \delta(f, 1) = f$.
 - Pour tout $i \in \llbracket 1, n \rrbracket$, $\delta(q_{i,m+1}, 0) = \delta(q_{i,m+1}, 1) = f$.
 - Pour tous $i \in \llbracket 1, n \rrbracket$ et $j \in \llbracket 1, m \rrbracket$,

$$\delta(q_{i,j}, 1) = \begin{cases} f & \text{si le littéral } x_j \text{ apparaît dans } c_i \\ q_{i,j+1} & \text{sinon} \end{cases},$$

$$\delta(q_{i,j}, 0) = \begin{cases} f & \text{si le littéral } \overline{x_j} \text{ apparaît dans } c_i \\ q_{i,j+1} & \text{sinon} \end{cases}.$$

22. Représenter la machine associée à la formule F_1 .
23. Donner une distribution de vérité $(v_1, v_2, v_3, v_4) \in \llbracket 0, 1 \rrbracket^4$ (la valeur v_i étant associée à la variable x_i) satisfaisant F_1 . Le mot $v_1 v_2 v_3 v_4$ est-il synchronisant ?
24. Montrer que tout mot u de longueur $m + 1$ est synchronisant. À quelle condition sur les $q_{i,1}.u$ un mot de longueur m est-il synchronisant ?
25. Montrer que si la formule F est satisfiable, toute distribution de vérité la satisfaisant donne un mot synchronisant de longueur m pour l'automate.
26. Inversement, prouver que si l'automate dispose d'un mot synchronisant de longueur au plus m , F est satisfiable. Donner alors une distribution de vérité convenable.

IV – Existence

On reprend le problème de l'existence d'un mot synchronisant pour une machine $M = (Q, \Sigma, \delta)$. Pour toute partie E de Q et tout mot u de Σ^* , on note $E.u = \{q.u \mid q \in E\}$.

27. Soit u un mot synchronisant de M et $u_0, u_1, \dots, u_r$ une suite de préfixes de u rangés dans l'ordre croissant de leur longueur et telle que $u_r = u$. Que peut-on dire de la suite des cardinaux $|Q.u_i|$?
28. Montrer qu'il existe un mot synchronisant si et seulement s'il existe pour tout couple d'états (q, q') de Q^2 un mot $u_{q,q'}$ tel que $q.u_{q,q'} = q'.u_{q,q'}$.

On veut se servir du critère établi ci-dessus pour déterminer s'il existe un mot synchronisant. On associe à M la machine $\tilde{M} = (\tilde{Q}, \Sigma, \tilde{\delta})$ définie par :

- $\tilde{Q}$ est formé des parties à un ou deux éléments de Q .
- $\forall (E, x) \in \tilde{Q} \times \Sigma, \tilde{\delta}(E, x) = \{\delta(q, x) \mid q \in E\}$.

29. Si $n = |Q|$, que vaut $\tilde{n} = |\tilde{Q}|$?

Pour la modélisation informatique, $\tilde{Q}$ doit être modélisé par l'intervalle $\llbracket 0, \tilde{n} - 1 \rrbracket$. Soit φ_n une bijection de $\tilde{Q}$ sur cet intervalle. On définit un type pour représenter les parties à un ou deux éléments :

```
type un_ou_deux = Un of int | Deux of int * int
```

On suppose qu'on dispose d'une fonction `set_to_nb` de signature `int -> un_ou_deux -> etat` telle que `set_to_nb n l` renvoie :

$$\begin{cases} \varphi_{n(\{i\})} & \text{si } l = \text{Un } i, \ 0 \leq i \leq n - 1 \\ \varphi_{n(\{i,j\})} & \text{si } l = \text{Deux } (i, j), \ 0 \leq i < j \leq n - 1 \end{cases}$$

On dispose aussi d'une fonction réciproque `nb_to_set` de type `int -> etat -> un_ou_deux` telle que `nb_to_set n q`, pour $n \in \mathbb{N}^*$ et $q \in \llbracket 0, \tilde{n} - 1 \rrbracket$, renvoie une valeur `Un i` ou `Deux (i, j)` (avec $i < j$) correspondant à $\varphi_n^{-1}(q)$. Ces deux fonctions sont supposées agir en temps constant. Pour ne pas confondre un état de $\tilde{Q}$ avec sa représentation informatique par un entier, on notera $\bar{q}$ l'entier associé à l'état q .

30. Écrire une fonction `delta2` de signature `machine -> etat -> lettre -> etat` qui, prenant en entrée une machine M , un état $\bar{q}$ de $\tilde{Q}$ et une lettre x , renvoie l'état de $\tilde{Q}$ atteint en lisant la lettre x depuis l'état q dans $\tilde{M}$.
31. À la machine $\tilde{M}$, on associe un graphe d'automate $\tilde{G}$ dont l'ensemble des sommets est $\tilde{Q}$ et dont l'ensemble des arcs est $\{(q, x, \tilde{\delta}(q, x)) \mid (q, x) \in \tilde{Q} \times \Sigma\}$. On associe à $\tilde{G}$ le graphe retourné $\tilde{G}_R$ qui a les mêmes sommets mais dont les arcs sont retournés : (q, x, q') est un arc de $\tilde{G}_R$ si et seulement si (q', x, q) est un arc de $\tilde{G}$. Écrire une fonction `retourne_machine` de signature `machine -> (etat * lettre) list array` qui, à partir d'une machine M , calcule le tableau `v` des listes d'adjacence du graphe $\tilde{G}_R$.
32. Justifier qu'il suffit d'appliquer la fonction `accessibles` de la partie II au graphe $\tilde{G}_R$ et à l'ensemble des sommets correspondant à des singletons pour déterminer si la machine M possède un mot synchronisant.
33. Écrire une fonction `existe_synchronisant` de signature `machine -> bool` qui dit si une machine possède un mot synchronisant.